



Authorization, RBAC, dan IDOR

Materi ini membahas authorization, perbedaan dengan authentication, konsep RBAC, risiko IDOR, dan proteksi endpoint Golang Gin.

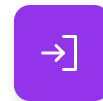


Juara Coding

June 2026

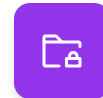
Authentication vs Authorization

Authentication memastikan identitas pengguna, sedangkan authorization menentukan hak akses setelah identitas terverifikasi.



Authentication

Menjawab siapa pengguna yang mengakses sistem, seperti login dan validasi token.



Authorization

Menjawab apa yang boleh dilakukan pengguna setelah identitasnya terverifikasi.



Di Backend Gin

Authorization umumnya berjalan setelah middleware authentication memverifikasi identitas pengguna.





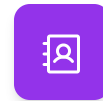
Mengapa Authorization Penting

Authorization mencegah akses ke data, fitur, dan aksi yang bukan hak pengguna, sehingga menjaga kerahasiaan, integritas, dan kontrol akses aplikasi.

Konsep Dasar RBAC



RBAC membatasi akses berdasarkan peran, sehingga permission dikelola per role, bukan per pengguna.



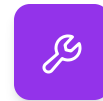
Berbasis Peran

Akses ditentukan oleh role seperti admin, staff, dan user dalam sistem.



Permission Berbeda

Setiap role memiliki izin berbeda untuk resource sesuai tanggung jawabnya.



Mudah Dikelola

Aturan akses diberikan ke role agar pengelolaan lebih konsisten dan efisien.

Struktur RBAC di Backend Gin

01 Data user, role, dan permission

RBAC mengelola hubungan antara user, role, dan permission untuk menentukan hak akses di backend.

03 Endpoint memeriksa izin aksi

Endpoint mengecek apakah role memiliki permission untuk aksi seperti read, create, update, atau delete.

02 Role disimpan ke context Gin

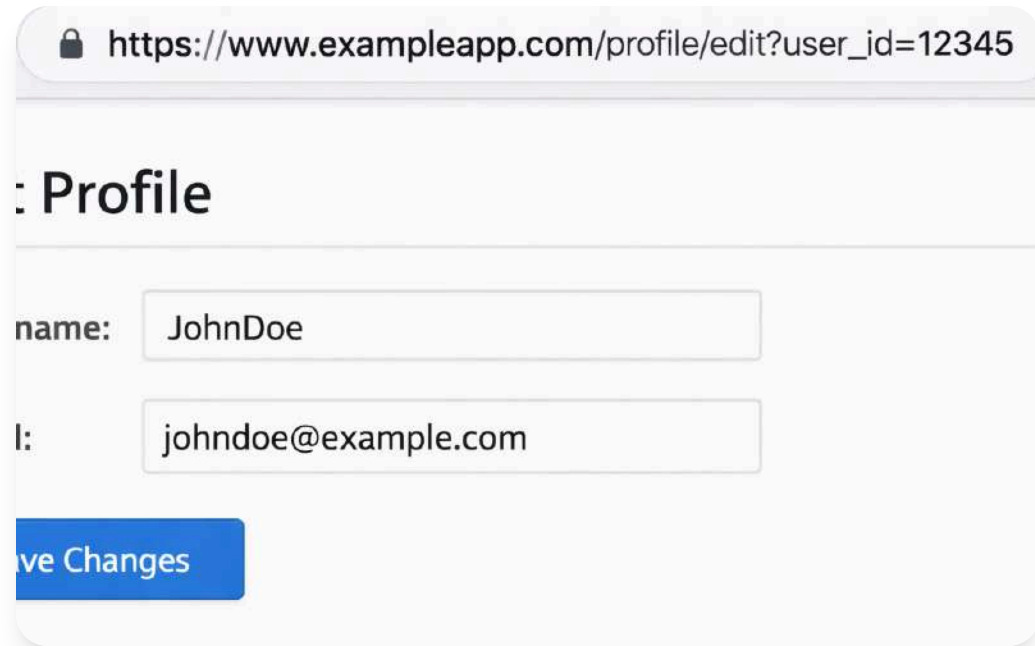
Setelah token dibaca di middleware authentication, informasi role dimasukkan ke context agar bisa dipakai endpoint.



Contoh Mapping Role dan Akses

Role	Lihat	Tambah	Ubah	Hapus
Admin	Ya	Ya	Ya	Ya
Staff	Ya	Terbatas	Terbatas	Tidak
User Biasa	Data sendiri		Tidak	Tidak

Mapping role dan akses harus jelas agar aturan otorisasi tetap konsisten di seluruh endpoint backend.



A screenshot of a web browser window. The address bar shows the URL: `https://www.exampleapp.com/profile/edit?user_id=12345`. The page title is "Profile". Below the title, there are two input fields. The first is labeled "name:" and contains the text "JohnDoe". The second is labeled "l:" and contains the text "johndoe@example.com". At the bottom left, there is a blue button with the text "Save Changes".

Pengenalan IDOR

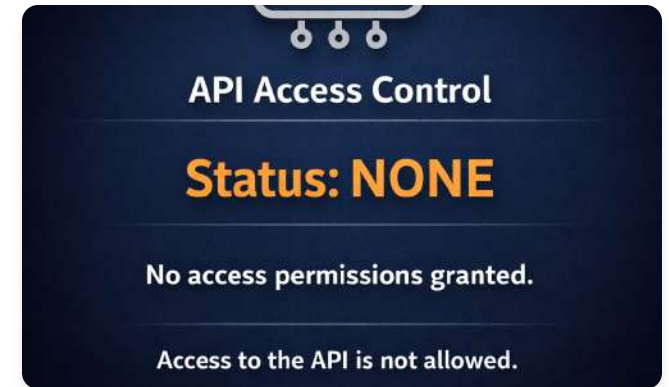
IDOR muncul saat ID langsung dipakai tanpa cek akses, sehingga data orang lain bisa diakses.



Contoh Kasus IDOR

Endpoint `GET /orders/123` bisa bocor jika backend hanya cek ID tanpa validasi pemilik order atau role pengguna.

Cara Mencegah IDOR



01 Jangan Andalkan ID Saja

ID resource tidak boleh jadi satu-satunya dasar akses karena bisa ditebak atau dimanipulasi.

02 Verifikasi Kepemilikan

Saat mengambil order, cocokkan *order.user_id* dengan user ID dari token autentikasi.

03 Tolak Akses Tidak Sah

Jika tidak cocok dan pengguna bukan admin, kembalikan respons *forbidden* atau *not found*.

Middleware Authorization di Gin



01 Aturan akses terpusat

Authorization middleware memusatkan pengecekan hak akses agar aturan endpoint tetap konsisten dan mudah dikelola.

03 Kode lebih rapi

Pendekatan ini membantu pemeliharaan aplikasi saat jumlah endpoint bertambah dan kebutuhan akses makin beragam.

02 Pemeriksaan setelah autentikasi

Setelah token divalidasi oleh authentication middleware, role pengguna diperiksa sebelum request masuk ke handler.

```
func RBACMiddleware(roles []string) mux.MiddlewareFunc {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            userRole, ok := r.Context().Value("role").(string)
            if !ok {
                http.Error(w, "Forbidden: No role assigned", http.StatusForbidden)
                return
            }
            authorized := false
            for _, role := range roles {
                if role == userRole {
                    authorized = true
                }
            }
            if !authorized {
                http.Error(w, "Forbidden: Access denied", http.StatusForbidden)
                return
            }
        })
    }
}
```

Contoh Middleware RBAC

Middleware ini membatasi akses endpoint hanya untuk pengguna dengan role tertentu yang diizinkan.

```
import ("github.com/gin-gonic/gin")

func AuthMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        // Middleware logic here
        c.Next()
    }
}

func main() {
    r = gin.Default()
    r.Use(AuthMiddleware(i))
    r.GET("/secure", SecureHandler)
    r.Run(":8080")
}
```

Middleware pada Route

Contoh penggunaan middleware pada route Gin: grup admin memakai autentikasi dan role *admin*, sementara profile hanya autentikasi.

Validasi Akses di Handler

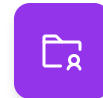


RBAC tidak cukup mencegah IDOR. Handler harus memastikan resource benar-benar milik pengguna yang sedang login.



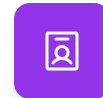
RBAC Bukan Jaminan

User yang valid belum tentu berhak mengakses semua resource meski role-nya sesuai.



Cek Kepemilikan Resource

Handler wajib memverifikasi bahwa data yang diminta memang terkait dengan pengguna aktif.



Waspada Parameter ID

Endpoint dengan parameter ID paling rentan jika akses hanya bergantung pada autentikasi atau role.

```
// Check if user is authenticated
user, err := getUserFromRequest(r)
if err := nil || \user.IsAuthenticated
    http.Error(w, "Unauthorized", http.StatusUnauthorized)
    return
}

// Check if user has required role
if \user.HasRole("admin"){
    http.Error(w, "Forbidden", http.StatusForbidden )
    return
}

// Protected content

w.Write([]byte("Welcome to the secure area"))
```

Contoh Proteksi IDOR

Handler memeriksa kepemilikan order dan role admin sebelum mengembalikan data, sehingga akses langsung ke objek dibatasi dengan aman.

Praktik Terbaik Authorization

||**

Simpan identitas di context

Ambil identitas pengguna dari token dan simpan ke context secara aman untuk dipakai di seluruh alur request.

× 5 ×

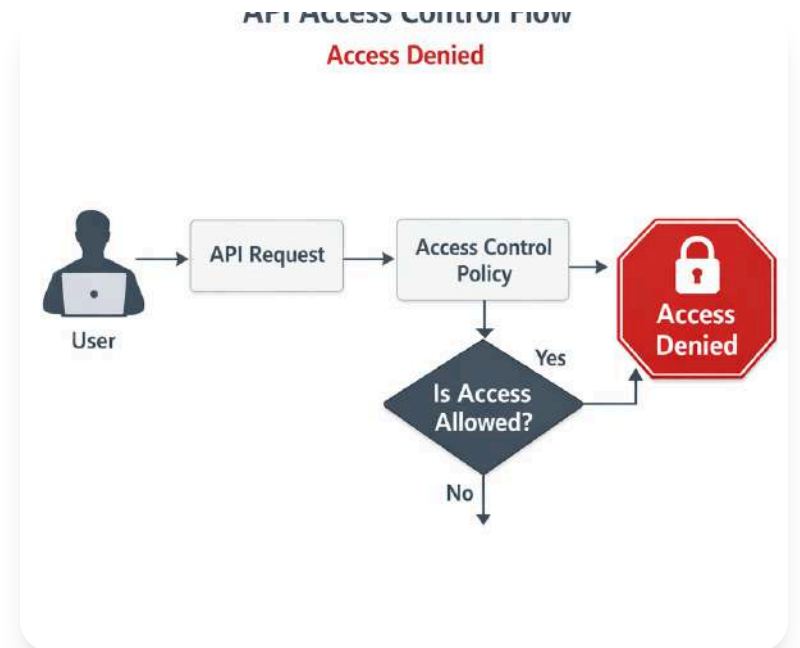
Cek aturan secara konsisten

Gunakan middleware untuk aturan umum dan cek kepemilikan resource di handler atau service tanpa hardcode berulang.

||**

Pisahkan auth dan authorization

Bedakan verifikasi identitas dengan aturan hak akses agar logika lebih jelas dan mudah dirawat.



Kesalahan Umum



Cek login saja tidak cukup

Hanya memastikan user sudah login tanpa memeriksa izin spesifik dapat membuka celah authorization.



Logika akses tersebar

Menaruh pemeriksaan akses di banyak tempat tanpa struktur jelas meningkatkan risiko IDOR dan inkonsistensi kontrol.

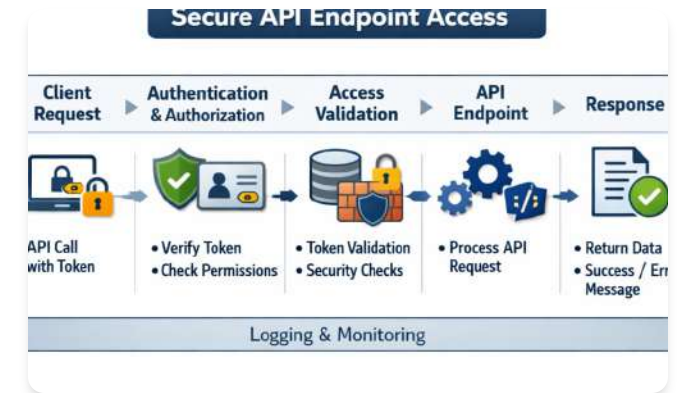


Ownership resource tidak diverifikasi

Akses ke data atau objek harus selalu dicek agar user hanya dapat membuka resource miliknya.



Alur Aman Akses Endpoint



01 Login dan token

Pengguna login lebih dulu, lalu sistem menerbitkan token sebagai dasar autentikasi untuk akses berikutnya.

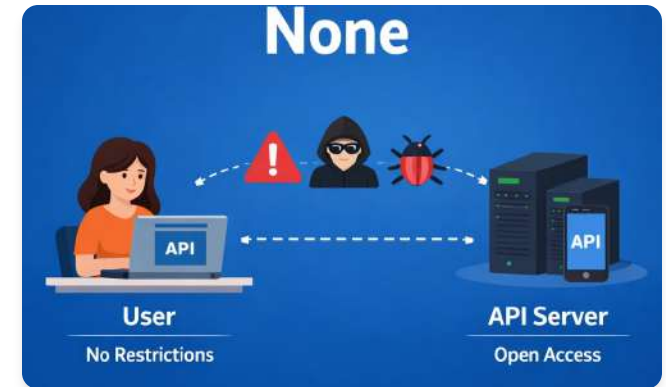
03 Role dan resource

Endpoint memeriksa role bila perlu, lalu memvalidasi kepemilikan resource sebelum data dikembalikan atau diubah.

02 Middleware dan context

Middleware memverifikasi token, lalu menyimpan user ID dan role ke context agar bisa dipakai di proses selanjutnya.

Latihan Praktik 1



01 GET dan PUT Artikel

Buat endpoint GET `/articles/:id` dan PUT `/articles/:id` untuk melihat dan mengubah artikel.

03 Cegah IDOR

Gunakan middleware authentication, role bila perlu, dan validasi ownership untuk mencegah akses ID tidak sah.

02 Aturan Akses Role

Admin dapat mengakses semua artikel, sedangkan user hanya boleh melihat dan mengubah artikel miliknya sendiri.

Latihan Praktik 2

API Access Requests

User	Requested Access	Request Date	Status	Actions
j. doe	Read / Write	2023-12-10	Pending	Approve Deny
s. kumar	Read Only	2023-12-08	Approved	Revoke
m. taylor	Full Access	2023-12-07	Pending	Approve Deny
a. chen	Read Only	2023-12-05	Denied	View
L. ngeyen	Write Only	2023-12-04	Approved	Revoke

Access Control Settings

Default API Access

No Access

Multi-Factor Authentication

☒

IP Whitelisting

5 Allowed IPs

Access Logs

View Logs

Active API Users

User	Access Level	API Key	Last Activity	Status	Active
r. smith	Read / Write	a9c4d...7321	2 hours ago	Active	Active
k. johnson	Read Only	b7e2f...d590	Today	Today	Active
m. lee	Full Access	c1245...8f9a	2023-12-15	Suspended	Suspended

01 Endpoint yang dibuat

Implementasikan GET /users, GET /users/:id, dan DELETE /users/:id pada modul manajemen user.

03 Pengujian akses

Uji skenario akses valid dan akses terlarang untuk memastikan kontrol akses berjalan sesuai kebutuhan.

02 Aturan otorisasi

Hanya admin boleh melihat semua user dan menghapus user. User biasa hanya boleh melihat profil miliknya sendiri.



Ringkasan Pertemuan 14

Authorization membatasi aksi sesuai izin. **RBAC** mengatur akses berbasis role, dan ownership mencegah **IDOR** pada resource.